Data Structures (CS212) Student Revision Sheet

AASTMT · Data Structures & Algorithms · concise notes, worked examples & practice

Complexity | Linked Lists | Stacks | Queues | Sorting | Trees / BST

🎯 Master complexity cheat-table

Structure / Algorithm	Access / Search	Insert	Delete	Space	Notes
Array	`O(1)` idx / `O(n)` search	`O(n)`	`O(n)`	`O(n)`	Contiguous → direct offset = O(1) index
Singly Linked List	`O(n)`	head `O(1)` · tail `O(n)`	head `O(1)` · mid `O(n)`	`O(n)`	No random access; 1 pointer/node
Doubly Linked List	`O(n)`	head/tail `O(1)`	given ptr `O(1)`	`O(n)`	+1 pointer/node; bidirectional
Stack (push/pop/top)	top `O(1)`	`O(1)`	`O(1)`	`O(n)`	LIFO; TOP does **not** scan
Circular Queue (addq/deleteq)	—	`O(1)`	`O(1)`	`O(n)`	FIFO; no shifting; 1 slot sacrificed
BST — balanced	`O(log n)`	`O(log n)`	`O(log n)`	`O(n)`	Skewed degrades to `O(n)`
Insertion Sort	best `Ω(n)`	worst/avg `O(n²)`		`O(1)`	Stable · in-place · no single Θ
Merge Sort	`θ(n log n)` — best = worst = avg			`O(n)`	Stable · **not** in-place (temp arrays)
Binary Search (sorted array)	`O(log n)`	—	—	`O(1)`	Halves the range each step

🧭 How to use this sheet

- Sheets 2–7 — one topic each: key facts, a complexity table, a **worked example**, and the traps that catch people.
- Sheets 8–9 — **mixed practice questions** with answers; cover the green answer and test yourself.
- Sheet 10 — method checklists (how to trace a stack/queue, build a BST, run a sort) + final exam-day tips.

⚠ The five ideas most often gotten wrong:

- Stack `TOP` and array index are `O(1)` (no scan); linked-list access is `O(n)`.
- Merge Sort's `O(n)` space is the **merge temp arrays**, not the recursion stack; it is **not** in-place.
- Circular queue: empty `front==rear`, full `front==(rear+1) mod n` — one slot is sacrificed.
- BST: depth(root)=0, height(leaf)=0; an inorder traversal of a BST is always ascending.
- Insertion Sort has no single `θ` (best `Ω(n)` ≠ worst `O(n²)`); Merge Sort is `θ(n log n)` always.

📈 Complexity & Asymptotic Notation

Asymptotic analysis describes **how an algorithm scales** as the input size `n` grows — a machine-independent way to compare algorithms. Forget seconds; measure the *growth rate* of the work. As `n` grows large, the dominant term swamps everything else, so constants and lower-order terms become irrelevant.

The Three Bounds

- **Big-O** `O(g(n))` — the *upper bound* (worst case / ceiling). "At most this bad." Performance will never exceed this.
- **Big-Omega** `Ω(g(n))` — the *lower bound* (best case / floor). "At least this much work" even on the friendliest input.
- **Big-Theta** `θ(g(n))` — the *tight bound* (sandwich). Exists **only when** the upper and lower bounds match, i.e. best case = worst case asymptotically.

Formal Definitions

- `f(n)=O(g(n))` if there exist constants `c>0` and `n0>0` such that `0 ≤ f(n) ≤ c·g(n)` for all `n ≥ n0`.
- `f(n)=Ω(g(n))` if `0 ≤ c·g(n) ≤ f(n)` for all `n ≥ n0`.
- `f(n)=θ(g(n))` *if and only if* `f(n)=O(g(n))` AND `f(n)=Ω(g(n))`.

The key phrase is **"for all n ≥ n0"**: the bound only has to hold beyond some threshold, not for tiny inputs. That is why a smaller term can lose even though it "wins" for small `n`.

Growth Ordering (slowest-growing = fastest)

`O(1)` < `O(log n)` < `O(n)` < `O(n log n)` < `O(n²)` < `O(2ⁿ)`

Notation	Name	Typical source
`O(1)`	Constant	Array index, hash lookup
`O(log n)`	Logarithmic	Binary search (halving)
`O(n)`	Linear	One simple loop / scan
`O(n log n)`	Linearithmic	Merge sort, heap sort
`O(n²)`	Quadratic	Two nested loops over `n`
`O(2ⁿ)`	Exponential	Enumerating all subsets

Simplification Rules

- **Drop constant factors:** `5n` → `O(n)`; `3n²` → `O(n²)`.
- **Drop lower-order terms:** keep only the fastest-growing term: `2n² + 9n + 11` → `O(n²)`.
- **Sequential code adds** (then keep the bigger): `O(n) + O(n²)` → `O(n²)`.
- **Nested loops multiply:** a loop of `O(n)` inside a loop of `O(n)` → `O(n²)`; an `O(log n)` inner step → `O(n log n)`.

Loop Invariants (proving correctness)

A loop invariant is a property true at the **start of every iteration**. Prove three things — the same structure as mathematical induction:

- **Initialization** — the invariant holds before the first iteration (the base case).
- **Maintenance** — if it holds before iteration `k`, it still holds before iteration `k+1` (the inductive step).
- **Termination** — when the loop ends, the invariant gives you the result you wanted to prove (the conclusion).

Single Θ vs. Best ≠ Worst

- If best, average, and worst case all have the **same growth**, the algorithm has a single `θ`. Example: an algorithm that does the same number of halve-and-merge steps regardless of input order is `θ(n log n)` always.
- If best and worst differ, there is **no single Θ** for the whole algorithm — you report best with `Ω` and worst with `O` separately. An input-sensitive algorithm (fast on nearly-sorted data, slow on reversed data) is the classic case.

Worked example: analyzing T(n) = 4n² + 30n + 12

Step 1 — identify the fastest-growing term: `4n²` beats `30n` and the constant `12` once `n` is large.

Step 2 — drop the constant factor and lower terms → growth class is `O(n²)`, in fact `θ(n²)`.

Step 3 — verify the formal Big-O. Pick `g(n)=n²`. For `n ≥ 30` we have `30n ≤ n²` and `12 ≤ n²`, so `4n² + 30n + 12 ≤ 4n² + n² + n² = 6n²`. Thus `c = 6`, `n0 = 30` satisfy `f(n) ≤ c·g(n)` for all `n ≥ n0`. Confirmed: `f(n) = O(n²)`.

Note: at `n = 1`, `f(1)=46` while `6·1²=6` — the bound fails for small `n`, which is exactly why the definition only requires `n ≥ n0`.

⚠ **Exam trap:** Big-O is an *upper* bound, not "the worst case." A correct (if loose) statement is `n = O(n²)`. Saying an algorithm "is `O(n²)`" only claims it is no worse than quadratic; it does not claim it always takes `n²` work. Use `θ` only when you can prove the upper and lower bounds match.

🔗 Linked Lists (SLL · DLL · CDLL)

A **linked list** is a linear data structure where each element is a **node** stored at a non-contiguous memory location. Unlike an array, the list grows and shrinks at runtime, and elements are reached by following pointers from the **head** rather than by index.

Singly Linked List (SLL)

- Node = `data` + one `next` pointer. The last node's `next` is `NULL`.
- A **head** pointer addresses the first node; traversal is **forward only** (no `prev`, no U-turns).
- **Insert at head** is `O(1)`: point new node's `next` to old head, then move head. *Order matters* — update head last.
- **Insert at tail without a tail pointer** is `O(n)`: you must walk the whole list to find the last node. A maintained tail pointer makes it `O(1)`.
- **Access / search by value** is `O(n)` — no random access, must follow the chain from head.
- **Delete** needs the *predecessor* node, so finding it is `O(n)`.
- Space = data + **1 pointer** per node.

Doubly Linked List (DLL)

- Node adds a `prev` pointer: each node knows both its successor and predecessor.
- Traversal works in **both directions** (forward via `next`, backward via `prev`).
- **Delete given a node pointer is `O(1)`**: rewire `node.prev.next = node.next` and `node.next.prev = node.prev` — no predecessor search needed.
- Head's `prev` = `NULL`; tail's `next` = `NULL`.
- Cost: **+1 pointer per node** (more memory, more edge cases — both links must stay consistent).

Circular Doubly Linked List (CDLL)

- The list forms a closed loop: `head.prev` points to the last node and `last.next` points back to head. There is **no `NULL`**.
- A single-node CDLL points to *itself* in both directions (`node.next == node`, `node.prev == node`).
- Traversal uses a **do-while loop**: process head first, then stop when `p == head`. A plain `while(p != head)` would skip the head because `p == head` is true at the start.
- Great for **cyclic iteration**: round-robin CPU scheduling, Alt+Tab window cycling.

Array vs Linked List & complexity

Operation	Array	SLL	DLL	CDLL
Access by index	`O(1)`	`O(n)`	`O(n)`	`O(n)`
Search by value	`O(n)`	`O(n)`	`O(n)`	`O(n)`
Insert at head	`O(n)`	`O(1)`	`O(1)`	`O(1)`
Insert at tail (no tail ptr)	`O(1)`	`O(n)`	`O(n)`	`O(1)`
Delete (node pointer known)	`O(n)`	`O(n)`	`O(1)`	`O(1)`
Space per element	data	+1 ptr	+2 ptrs	+2 ptrs

- **Array:** fast `O(1)` indexing and good cache locality, but fixed size and costly shifting on insert/delete.
- **Linked list:** dynamic size and cheap insert/delete at a known spot, but no random access and extra pointer storage.
- **Applications:** SLL → stacks & queues; **DLL → browser history and undo/redo** (back follows `prev`, forward follows `next`); CDLL → round-robin scheduling / Alt+Tab.

Worked example: insert at head of an SLL holding 7 → 3 → 9

Start: `head → [7] → [3] → [9] → NULL`. We insert a new node with value **5** at the head.

1. Allocate `p = new Node(5)`.
2. **Point new node to old head first:** `p.next = head` (so `p.next` → the node holding 7).
3. **Then move head:** `head = p`.

Result: `head → [5] → [7] → [3] → [9] → NULL`. Only one `next` pointer was set and no traversal happened, so this is `O(1)`. If we had done `head = p` *before* `p.next = head`, we would lose the rest of the list.

⚠ **Exam trap:** "Linked lists give `O(1)` access to the k-th element." False — access is `O(n)` because you must walk from the head; only arrays have `O(1)` indexing. Also remember `O(1)` delete in a DLL requires you to *already hold the node's pointer*; finding the node first is still `O(n)`.

📚 Stacks: Operations, Multi-Stack & Infix→Postfix

1. The Stack ADT — LIFO

- A stack is a list where insertion and deletion happen at **one end only**, called the **top**.
- Discipline is **LIFO** (Last In, First Out): the most recently pushed item is the first popped.
- Core operations: `push(x)` add to top, `pop()` remove & return top, `top()/peek()` read top without removing, `empty()`, `size()`.
- Array-based: bottom at `data[0]`, top at `data[top]`; an index variable `top` tracks the last filled slot.
- Every core operation is `O(1)` — no loop, no scan. `size()` returns `top+1` directly.

2. Overflow, Underflow & why TOP is O(1)

- **OVERFLOW**: pushing when the array is full. With capacity `n` and 0-based `top`, full means `top == n-1`; some texts test `top == n` when `top` counts items. Push first checks fullness, then `top←top+1; data[top]←x`.
- **UNDERFLOW**: popping/peeking an empty stack. Empty means `top == -1` (item-count convention: `top == 0`). Pop checks emptiness, returns `data[top]`, then `top←top-1`.
- `top()` reads `data[top]` in one step — it never scans the array, so it is `O(1)`, not `O(n)`.
- Overflow is a limitation of the *array* implementation, not of the Stack ADT; a linked-list stack pushes at the head and grows freely.

3. Multi-Stack: packing several stacks in one array

- **Two stacks, opposite ends:** stack 1 grows upward from index `0` (top index `top1`); stack 2 grows downward from index `n-1` (top index `top2`). Memory is shared, so neither has a fixed half.
- Collision (overall OVERFLOW) occurs only when the tops meet: `top1 + 1 == top2`. Until then either stack may keep growing.
- This is more space-efficient than two half-size arrays: one stack can use almost all `n` slots if the other is small.
- **Three or more stacks** in one array cannot use opposite ends. When a stack overflows into its neighbour's region, you must **shift neighbouring stacks** (and their boundaries) to make room — an expensive `O(n)` reorganisation, unlike the `O(1)` two-stack case.

4. Applications of stacks

- **Function/method calls:** the run-time call stack pushes a frame per call; returning pops it (supports recursion).
- **Undo** in editors: each action pushed; Undo pops the last action (LIFO matches "undo most recent").
- **Bracket / delimiter matching:** push each opener; on a closer, pop and check it matches. Balanced ⇔ every closer matches the popped opener AND the stack is empty at the end.
- **Expression handling:** infix→postfix conversion and postfix evaluation; browser page-visited (back) history.

5. Infix → Postfix using a stack (ICP vs ISP)

- Scan left→right. **Operand** → append straight to output. **Operator** → compare its incoming precedence **ICP** with the in-stack precedence **ISP** of the operator on top.
- While the stack top is an operator with `ISP(top) ≥ ICP(incoming)`, pop it to output; then push the incoming operator. This makes `* /` beat `+ -`, and equal-precedence left-associative operators pop the earlier one first.
- `(` → push it. `)` → pop to output until `(` is popped; discard both parentheses.
- **Exponent `^` is right-associative:** set `ICP(^) > ISP(^)`, so a new `^` does NOT pop the `^` already on the stack (e.g. `a^b^c → a b c ^ ^`).
- At end of input, pop all remaining operators to output.

Operator	ISP (on stack)	ICP (incoming)	Associativity
`(`	lowest (never pops others)	highest (always pushes)	—
`+ -`	low	low	left
`* /`	medium	medium	left
`^`	high	highest (above its own ISP)	right

Worked example: convert `p + q * r - s` to postfix

1. **p** → operand → output: `p`; stack: empty.
2. **+** → stack empty → push. Stack: `+`; output: `p`.
3. **q** → operand → output: `p q`.
4. ***** → top is `+`, and `ISP(+) < ICP(*)` → do NOT pop → push. Stack: `+ *`; output: `p q`.
5. **r** → operand → output: `p q r`.
6. **-** → top `*`: `ISP(*) ≥ ICP(-)` → pop `*` → output: `p q r *`. Top now `+`: `ISP(+) ≥ ICP(-)` (equal, left-assoc) → pop `+` → output: `p q r * +`. Stack empty → push `-`. Stack: `-`.
7. **s** → operand → output: `p q r * + s`.
8. End of input → pop remaining: `-` → output: `p q r * + s -`.

Result: `p q r * + s -`

⚠ **Exam trap:** `top()` is `O(1)` — it reads one indexed slot and does NOT scan the stack. Also: a closer that finds the *wrong* opener (or an *empty* stack) means unbalanced, AND leftover openers at the end also mean unbalanced. Finally, `^` is right-associative, so `ICP(^) > ISP(^)`: a second `^` is pushed on top of an existing `^` rather than popping it.

Queues (Linear & Circular)

The Queue ADT & FIFO

- A queue stores objects under the **First-In First-Out (FIFO)** rule: the first element added is the first removed.
- **ADDQ / enqueue** inserts at the **rear**; **DELETEQ / dequeue** removes at the **front**. The two operations act on *opposite* ends.
- Two markers track the ends: `front` (first element) and `rear` (last element). Compare with a stack, which has a single `top` pointer.
- Auxiliary ops: `size()`, `empty()`, `front()` (peek first), `rear()` (peek last).
- **Overflow** = ADDQ on a full queue. **Underflow** = DELETEQ/front on an empty queue (throws Queue Empty).

Applications

- **Print spooler:** documents sent to a shared printer are queued and printed in arrival order — pure FIFO.
- Waiting lines / ticket counters, access to shared resources, multiprogramming and round-robin CPU scheduling.
- Used wherever **sequential processing** in arrival order is required.

Linear array queue: the drift problem

- Keeping `queue[0]` permanently as front forces every element to shift on each delete — wasteful.
- Instead we just advance `front` on delete and `rear` on add. But both indices keep **drifting** rightward.
- **False-full:** eventually `rear` reaches the last cell while early cells sit empty. The queue reports "full" although free space exists at the start — a wasted-slot illusion, not a real overflow.

Circular queue Q(0 : n-1)

Wrapping the indices with modulo reuses the freed front cells, eliminating false-full.

- **ADDQ:** `rear = (rear + 1) % n`, then store the value at `queue[rear]`.
- **DELETEQ:** `front = (front + 1) % n`, then read `queue[front]`.
- **Empty test:** `front == rear`.
- **Full test:** `front == (rear + 1) % n`.
- **One slot sacrificed:** because both empty and full would otherwise give `front == rear`, we deliberately leave one cell unused so the two states stay distinguishable. A queue of capacity `n` holds at most `n - 1` elements.
- **Count:** number of elements = `(rear - front + n) % n`.
- Both ADDQ and DELETEQ run in `O(1)` time — no shifting, ever.

Aspect	Linear array queue	Circular queue
Index update	`rear++` / `front++`	`(rear+1)%n` / `(front+1)%n`
Reuses freed cells	No	Yes (wrap-around)
False-full possible	Yes	No
Usable capacity	n	n − 1 (one sacrificed)
ADDQ / DELETEQ cost	`O(1)`	`O(1)`

Worked example: circular queue with n = 5

Cells indexed 0..4. Start `front = 0`, `rear = 0` (empty, since front == rear).

1. **ADDQ 7:** rear = (0+1)%5 = 1, store 7 → cell[1]=7.
2. **ADDQ 3:** rear = (1+1)%5 = 2, store 3 → cell[2]=3.
3. **ADDQ 9:** rear = (2+1)%5 = 3, store 9 → cell[3]=9.
4. **ADDQ 4:** rear = (3+1)%5 = 4, store 4 → cell[4]=4. Now front=0, rear=4.
5. **Full check:** (rear+1)%5 = (4+1)%5 = 0 = front → **FULL**. Holds 4 = n−1 elements, one cell (index 0) sacrificed.
6. **DELETEQ:** front = (0+1)%5 = 1, read cell[1] = **7** (FIFO: 7 entered first). Now front=1, rear=4.
7. **Count:** (rear − front + n) % n = (4 − 1 + 5) % 5 = 8 % 5 = **3** elements remain (3, 9, 4).
8. **ADDQ 6:** rear = (4+1)%5 = 0, store 6 → cell[0]=6. The rear has **wrapped around** into the freed cell, which a linear queue could not do.

⚠ **Exam trap:** In ADDQ/DELETEQ the index is advanced *before* the store/read, not after. Also remember a capacity-`n` circular queue holds only `n − 1` items: the full test `front == (rear+1)%n` deliberately sacrifices one slot so that "full" and "empty" do not both look like `front == rear`.

🔢 Sorting: Insertion Sort & Merge Sort

Two foundational sorting algorithms. **Insertion Sort** is simple and adaptive but quadratic in the worst case; **Merge Sort** is a divide-and-conquer algorithm with a guaranteed `θ(n log n)` running time at the cost of extra memory.

Insertion Sort — the card player

- **Idea:** grow a **sorted prefix** on the left. Each pass takes the next element as the **key**, then **shifts every larger element one slot to the right** until the key's correct slot opens up, and drops it in.
- **Loop shape:** outer loop `j = 1 .. n-1` (key = arr[j]); inner `while` walks left while `arr[i] > key`, copying arr[i] into arr[i+1].
- **Best case** (already sorted): inner loop never runs — one comparison per element → `Ω(n)`.
- **Worst case** (reverse sorted): every key travels to index 0 → `1+2+...+(n-1) = n(n-1)/2` comparisons → `θ(n²)`.
- **Stable** (uses `>`, never moves equal keys past each other) and **in-place** → `O(1)` extra space.
- **Adaptive & online:** fast on nearly-sorted data; can sort items as they arrive.

Merge Sort — divide & conquer

- **Divide:** split the array in half recursively until each subarray is a single element (already sorted by definition).
- **Conquer / Merge:** "zip" two sorted halves into one by repeatedly comparing front elements and copying the smaller first.
- **Recurrence:** `T(n) = 2T(n/2) + n`. The tree is `log n` levels deep; each level does `θ(n)` merge work → `θ(n log n)` in **best, worst, and average** cases (input-agnostic).
- **Not in-place:** merging needs an auxiliary buffer → `O(n)` extra space.
- **Stable** when the merge uses `<=` to keep the left element on ties.

Which to use?

- Small or nearly-sorted arrays → **Insertion Sort** (tiny constants, adaptive, no extra memory).
- Large arrays where worst-case speed matters → **Merge Sort** (guaranteed `θ(n log n)`, stable).
- Hybrids (e.g. Timsort) combine both: Insertion Sort on small runs, Merge Sort to combine them.

Property	Insertion Sort	Merge Sort
Best case	`Ω(n)`	`θ(n log n)`
Worst / Average	`θ(n²)`	`θ(n log n)`
Space	`O(1)` (in-place)	`O(n)` (aux buffer)
Stable	Yes	Yes
Adaptive	Yes	No
Strategy	Incremental	Divide & conquer

Worked example: Insertion Sort on [7, 3, 9, 1]

Sorted prefix shown in **bold**; key in *italics*.

1. Start: [**7**, *3*, 9, 1]. key=3. 7>3 → shift 7 right → insert 3 → [**3, 7**, 9, 1].
2. key=9. 7>9? No → 9 stays → [**3, 7, 9**, 1].
3. key=1. 9>1 shift, 7>1 shift, 3>1 shift, hit left end → insert 1 at index 0 → [**1, 3, 7, 9**].

Result: `[1, 3, 7, 9]`. The last pass did 3 comparisons + 3 shifts because 1 had to travel the full distance — exactly the costly pattern a reverse-sorted array repeats on every key.

Merge Sort sketch on the same array: split [7,3,9,1] → [7,3] and [9,1] → singletons [7][3][9][1]. Merge: [7],[3]→[3,7]; [9],[1]→[1,9]. Final merge of [3,7] and [1,9]: compare 3 vs 1 → 1; 3 vs 9 → 3; 7 vs 9 → 7; then 9 → `[1, 3, 7, 9]`.

⚠ **Exam trap:** Merge Sort is `θ(n log n)` in **all** cases — a sorted input does **not** make it faster (no "best case = n"). That adaptive best-case `Ω(n)` belongs to **Insertion Sort**. Also: Merge Sort is NOT in-place (it needs `O(n)` aux); Insertion Sort is the in-place / `O(1)`-space one.

🌳 Trees & Binary Search Trees

Tree Terminology

- **Root**: the single topmost node, has no parent. Every tree has exactly one.
- **Parent / Child**: a node directly above / below another, connected by an **edge**.
- **Leaf**: a node with no children (sits at the end of a branch).
- **Internal node**: any node that has at least one child.
- **Subtree**: a node plus all of its descendants — every node roots its own subtree.
- A tree with `N` nodes has exactly `N-1` edges.

Depth vs Height (the classic confusion)

- **Depth of a node** = number of edges from the *root down* to that node. So **depth(root) = 0**.
- **Height of a node** = number of edges on the longest path from that node *down* to a leaf. So **height(leaf) = 0**.
- **Height of the whole tree = height of the root**.
- Depth grows downward; height is measured upward from the bottom.

Binary Search Tree (BST) Ordering Rule

- For **every** node: `left subtree < node < right subtree` — not just at the root.
- **Insert / Search**: start at root, go **left** if target < node, **right** if target > node, repeat.
- Reaching `NULL` during a search means the value is **absent**.
- The first value inserted becomes the root; later values find their slot via the rule.

Performance

Shape	Search / Insert	Why
Balanced BST	`O(log n)`	Each comparison eliminates ~half the remaining nodes
Skewed BST (sorted input)	`O(n)`	Degenerates into a linked list — one long chain

- Inserting *already-sorted* values (e.g. 1,2,3,4,5) builds a fully skewed tree — the worst case.

Traversals (Depth-First)

- **Preorder (Root, Left, Right)**: root recorded first → used to **copy / serialize** a tree.
- **Inorder (Left, Root, Right)**: on a BST this outputs values in **ascending sorted order**.
- **Postorder (Left, Right, Root)**: children visited before parent → **safe deletion** / freeing memory.
- All three are recursive with a base case `if node == NULL: return`; only the position of the "visit" step moves.

Worked example: build a BST and traverse it

Insert this order into an empty BST: **50, 25, 70, 10, 35, 60**.

1. 50 → root.
2. 25 < 50 → left of 50.
3. 70 > 50 → right of 50.
4. 10: 10<50 left → 10<25 left → left child of 25.
5. 35: 35<50 left → 35>25 right → right child of 25.
6. 60: 60>50 right → 60<70 left → left child of 70.

Resulting tree: root 50; left subtree (25 with children 10, 35); right subtree (70 with left child 60).
Inorder (L,Root,R) = 10, 25, 35, 50, 60, 70 → ascending sorted, confirming a valid BST.
Preorder (Root,L,R) = 50, 25, 10, 35, 70, 60.
Postorder (L,R,Root) = 10, 35, 25, 60, 70, 50 → root 50 comes last, safe for deletion.
Search 35: 35<50 go left → 35>25 go right → found. Only 3 nodes visited. **Search 42**: 42<50 left → 42>25 right → 42>35 right → child is `NULL` → absent.
Depth/Height: depth(50)=0, depth(25)=1, depth(10)=2. Height(leaf 10)=0, height(25)=1, height of tree = height(50) = 2.

⚠ **Exam trap:** Depth and height are mirror images — **depth(root)=0** (count down from root) while **height(leaf)=0** (count up to a leaf). Also remember **Inorder** (not Preorder) is the one that prints a BST in sorted order, and a BST built from sorted input is skewed, giving `O(n)`, not `O(log n)`.

✍ Mixed Practice — Test Yourself

Cover the green ✓ answer and attempt each question first. Same skills as the exam, fresh questions.

Complexity

1. [MCQ] Simplify $T(n) = 7n^3 + 200n^2 + 5000n + 99$ to its tightest growth class.
a) $O(n^2)$ **b)** $O(n^3)$ **c)** $O(n \log n)$ **d)** $O(n^4)$
✓ B) $O(n^3)$

2. [TF] Big-Theta Θ(g(n)) exists for an algorithm only when its best-case and worst-case growth rates match (i.e. it is both O(g(n)) and Ω(g(n))).
✓ TRUE

3. [FILL] In a loop-invariant correctness proof, the three properties that must be shown are Initialization, _____, and Termination.
✓ Maintenance

4. [MCQ] Which list places the growth classes in correct order from SLOWEST-growing to FASTEST-growing?
a) $O(n^2) < O(n \log n) < O(n) < O(\log n) < O(1)$ **b)** $O(1) < O(n) < O(\log n) < O(n \log n) < O(2^n)$ **c)** $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$ **d)** $O(\log n) < O(1) < O(n \log n) < O(n) < O(2^n)$
✓ C) $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$

5. [TRACE] Given $f(n) = 2n^2 + 6n$, find the SMALLEST integer constant c that makes $f(n) \leq c \cdot n^2$ hold for all $n \geq 1$ (use $g(n)=n^2$, $n0=1$). Show the resulting c.
✓ c = 8

Linked Lists

6. [MCQ] What does a singly linked list node store?
a) Only the data value **b)** Data plus a pointer to the next node **c)** Data plus pointers to both the next and previous nodes **d)** Data plus the index of the element
✓ B) Data plus a pointer to the next node

7. [TF] In a singly linked list, accessing the element at a given index takes O(1) time, just like an array.
✓ FALSE

8. [FILL] In a doubly linked list, each node stores its data, a next pointer, and a _____ pointer that references the previous node.
✓ prev

9. [MCQ] Inserting a node at the tail of a singly linked list that has only a head pointer (no tail pointer) has what time complexity?
a) O(1) **b)** O(log n) **c)** O(n) **d)** O(n^2)
✓ C) O(n)

10. [TRACE] An SLL is head → [4] → [6] → [2] → NULL. You insert a node with value 9 at the head, then delete the node containing value 6. Write the final list from head to NULL (values in order).
✓ 9 -> 4 -> 2 -> NULL

Stacks

11. [MCQ] An empty stack is popped using a 0-based top index where empty is represented by top == -1. What error condition does this attempted pop trigger?
a) Overflow **b)** Underflow **c)** Collision **d)** Out-of-order access
✓ B) Underflow

12. [TF] In a stack, both insertion (push) and deletion (pop) occur only at the top end, giving Last-In-First-Out (LIFO) behaviour.
✓ TRUE

13. [FILL] The error that occurs when push() is called on a stack whose array is already full is called _____.
✓ overflow

14. [MCQ] A stack of capacity n stores its item count in a variable top (top counts the elements present). Which test correctly detects that the stack is FULL before a push?
a) top == -1 **b)** top == 0 **c)** top == n **d)** top == n + 1
✓ C) top == n

15. [TRACE] Trace the infix-to-postfix conversion of (m + n) * k using an operator stack and give the final postfix expression.
✓ m n + k *

✍️ Mixed Practice — Test Yourself

Cover the green ✓ answer and attempt each question first. Same skills as the exam, fresh questions.

Queues

16. [MCQ] Which principle does a queue follow?
a) LIFO (Last-In First-Out) **b)** FIFO (First-In First-Out) **c)** Random access **d)** Priority by value
✓ B) FIFO (First-In First-Out)

17. [TF] In a queue, insertions (ADDQ) happen at the rear and deletions (DELETEQ) happen at the front.
✓ TRUE

18. [FILL] The reason one slot is intentionally left unused in a circular queue is so the ____ condition can be distinguished from the empty condition.
✓ full

19. [MCQ] In a circular queue of size n, what is the correct ADDQ (enqueue) update for the rear index before storing the new value?
a) rear = rear + 1 **b)** rear = (rear - 1) % n **c)** rear = (rear + 1) % n **d)** rear = (front + 1) % n
✓ C) rear = (rear + 1) % n

20. [TRACE] A circular queue has n = 4 (cells 0..3), starting front = 0, rear = 0 (empty). Perform: ADDQ 5, ADDQ 8, ADDQ 1, then one DELETEQ. Using rear= (rear+1)%n then store, and front=(front+1)%n then read, what value does the DELETEQ return and what are front and rear afterward?
✓ Returns 5; front = 1, rear = 3

Sorting

21. [MCQ] What input order produces the WORST-case running time for Insertion Sort?
a) Already sorted in ascending order **b)** Reverse (descending) sorted **c)** All elements equal **d)** Sorted except for the last element
✓ B) Reverse (descending) sorted

22. [TF] Insertion Sort runs in O(n) time on an array that is already sorted in ascending order.
✓ TRUE

23. [FILL] The variable that holds the element currently being inserted into the sorted prefix in Insertion Sort is conventionally called the ____.
✓ key

24. [MCQ] Which recurrence correctly describes the running time of Merge Sort?
a) T(n) = T(n-1) + n **b)** T(n) = 2T(n/2) + n **c)** T(n) = T(n/2) + 1 **d)** T(n) = 2T(n-1) + 1
✓ B) T(n) = 2T(n/2) + n

25. [TRACE] Trace Insertion Sort on the array [6, 2, 8, 4]. Give the array contents after each element has been inserted (i.e., after each outer-loop pass).
✓ [2,6,8,4] -> [2,6,8,4] -> [2,4,6,8]

Trees & BST

26. [MCQ] Which traversal order visits the root LAST, making it the safe choice for deleting/freeing a tree?
a) Preorder **b)** Inorder **c)** Postorder **d)** Level-order
✓ C) Postorder

27. [TF] In any tree, the depth of the root node is 0.
✓ TRUE

28. [FILL] In a BST, for every node the values in its ____ subtree are less than the node and the values in its right subtree are greater.
✓ left

29. [MCQ] A BST is built by inserting 40, 20, 60, 10, 30. Where does the value 25 go?
a) Left child of 40 **b)** Right child of 20 **c)** Left child of 30 **d)** Right child of 60
✓ C) Left child of 30

30. [TRACE] A BST is built by inserting 8, 3, 11, 1, 6, 14 into an empty tree. Give the INORDER traversal output.
✓ 1 3 6 8 11 14

🧰 Method Checklists & Exam-Day Strategy

Trace an array stack

- PUSH: check `top ≠ n` → `top++` → store.
- POP: check `top ≠ 0` → read `STACK[top]` → `top--`.
- TOP: return `STACK[top]`, no change.
- Track *top*, the contents, and the returned value at every step.

Trace a circular queue Q(0:n-1)

- ADDQ: `rear=(rear+1) mod n` → store at `Q[rear]`.
- DELETEQ: `front=(front+1) mod n` → read `Q[front]`.
- Empty `front==rear`; full `front==(rear+1) mod n`.
- Watch the wrap from index `n-1` back to `0`.

Build / read a BST

- Insert: smaller → go left, larger → go right, to an empty slot.
- Depth(root)=0; Height(leaf)=0; tree height = root height.
- Inorder = ascending; Preorder = copy; Postorder = safe delete.
- Balanced search ≈ $\log_2 n$ steps; skewed = n.

Run the two sorts

- Insertion: take key, shift larger elements right, drop key in. Per pass, A[0..j] becomes sorted.
- Merge: split to single elements, then merge adjacent sorted runs in order.
- Insertion best $O(n)$ /worst $O(n^2)$; Merge $\theta(n \log n)$ always, $O(n)$ extra space.

🗂 Traversal quick-reference

Traversal	Order	Use
Preorder	Root → L → R	Copy / serialize a tree
Inorder	L → Root → R	Ascending sorted output of a BST
Postorder	L → R → Root	Safely delete / free all nodes

⚠ On the day:

- For "choose the correct statement(s)" questions, check *every* clause — reject an option if any part is false.
- One wrong word flips a True/False (in-place, O(1), stable, NULL, FIFO/LIFO) — read carefully.
- Show your trace tables step-by-step; partial credit lives in the working, not just the final answer.
- State complexity with the right symbol: O (worst), Ω (best), θ (only when they match).